COM 682

Cloud Native Development

Cloud Design Patterns and Architecture

Dr Rashid Kamal

A key initial decision is which cloud architecture pattern to use.

What kind of architecture are you building?

We have identified several distinct architecture styles. There are benefits and challenges to each.

Cloud Architecture

An architecture style is a family of architectures that share certain characteristics.

For example, N-tier is a common architecture style.

More recently, microservice architectures have started to gain popularity.

Architecture styles don't require the use of particular technologies, but some technologies are well-suited for certain architectures. For example, containers are a natural fit for microservices.

Architecture styles as constraints

An architecture style **places constraints** on the design, including the set of elements that can appear and the allowed relationships between those elements.

Constraints guide the "shape" of an architecture by restricting the range of choices.

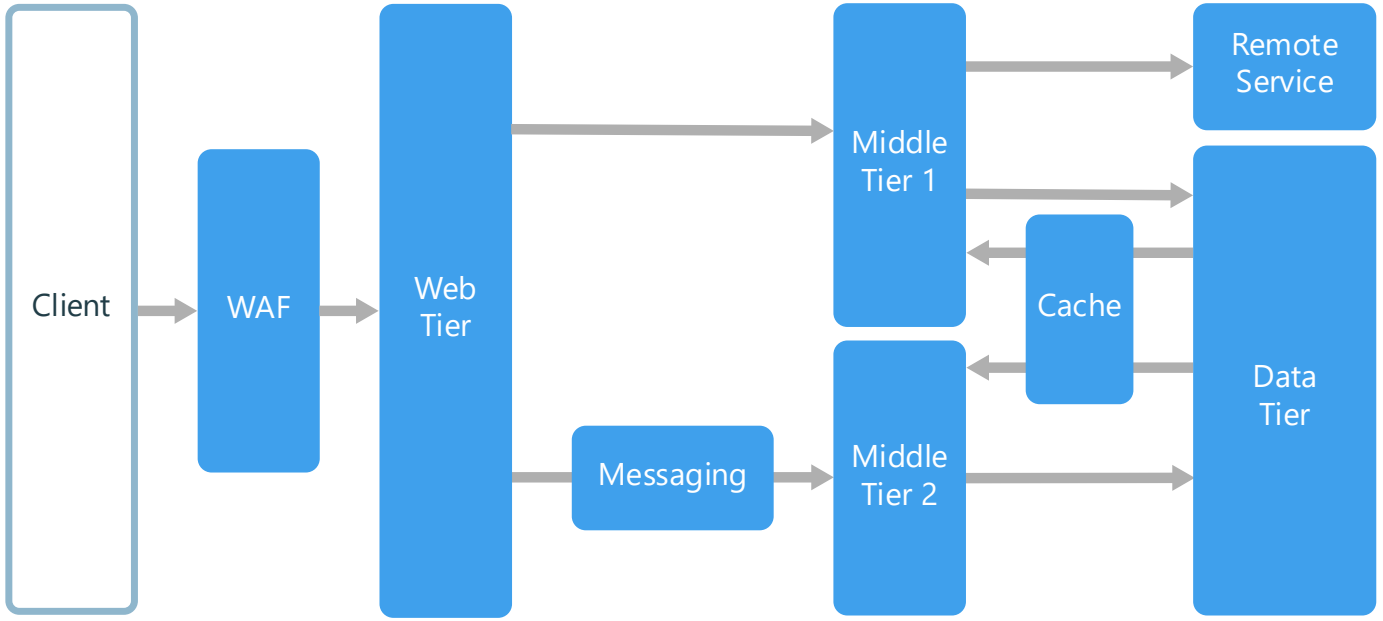
Before choosing an architecture style, **ensure you understand the underlying principles and constraints** **of that style**.

Otherwise, you can end up with a design that conforms to the style at a superficial level, but does not achieve the full potential of that style. It's also important to be realistic.

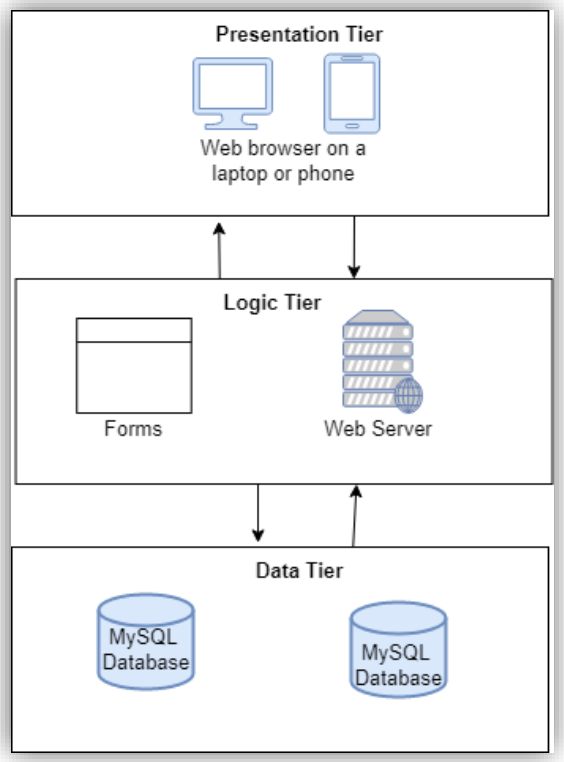
Cloud Architecture

There are a range of styles which may be adopted to inform a solution, in this lecture (and for CW) we will focus on the 7 main ones:

Cloud Architectures
1) N-tier
2) Web-Queue-Worker
3) Microservices
4) Event-driven architecture
5) Serverless Architecture



Web Application Firewall



N-Tier

An N-tier architecture divides an application into logical layers and physical tiers.

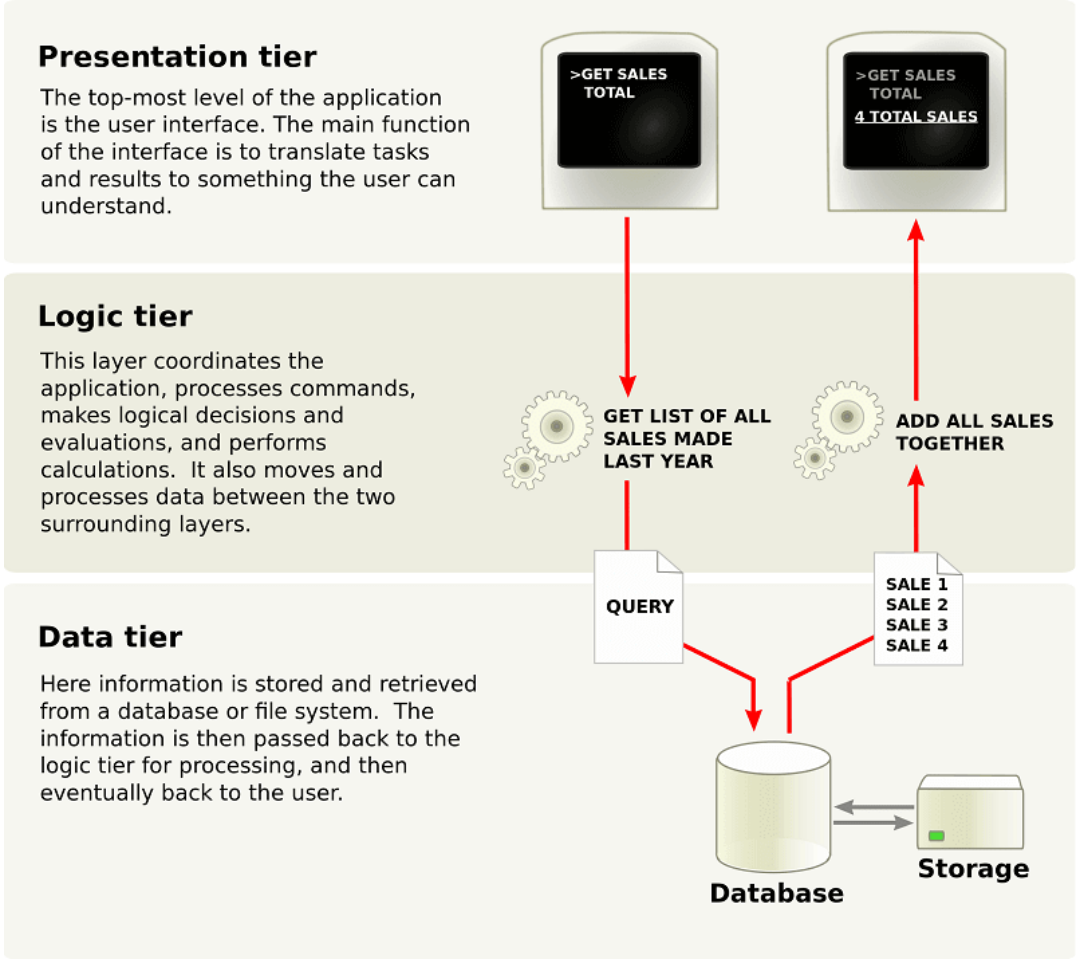
1. N-Tier Application

N-tier is a traditional architecture for enterprise applications.

Dependencies are managed by dividing the application/service into layers that perform logical functions, such as presentation, business logic, and data.

N-tier is a natural fit for migrating existing applications that already use a layered architecture.

N-tier is most often seen in infrastructure as a service (IaaS) solutions, or application that use a mix of IaaS and managed services.

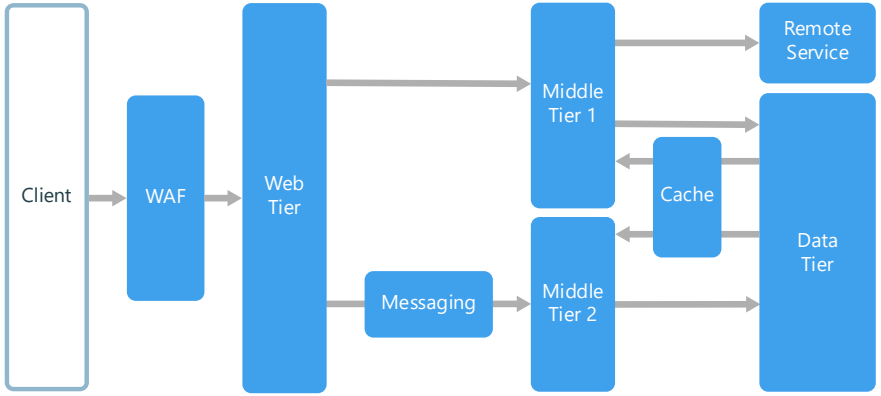


N-Tier

Layers are a way to separate responsibilities and manage dependencies. Each layer has a specific responsibility.

Tiers may be physically separated, running on separate machines.

A tier can call to another tier directly, or use asynchronous messaging (message queue). Although each layer might be hosted in its own tier, that's not required.

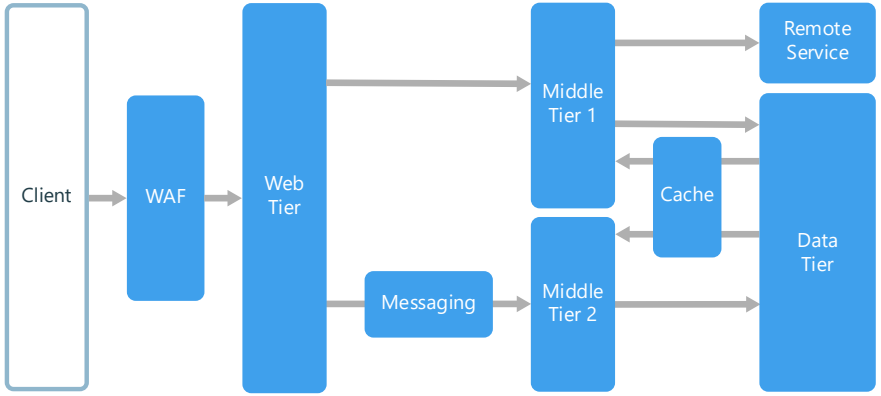


When to use this Architecture?

Consider an N-tier architecture for:

- **Simple web** applications.
- **Migrating** an on-premises application to Azure with minimal refactoring.
- **Unified** development of on-premises and cloud applications.

N-tier architectures are very common in traditional on-premises applications, so it's a natural fit for **migrating** existing workloads to Azure.

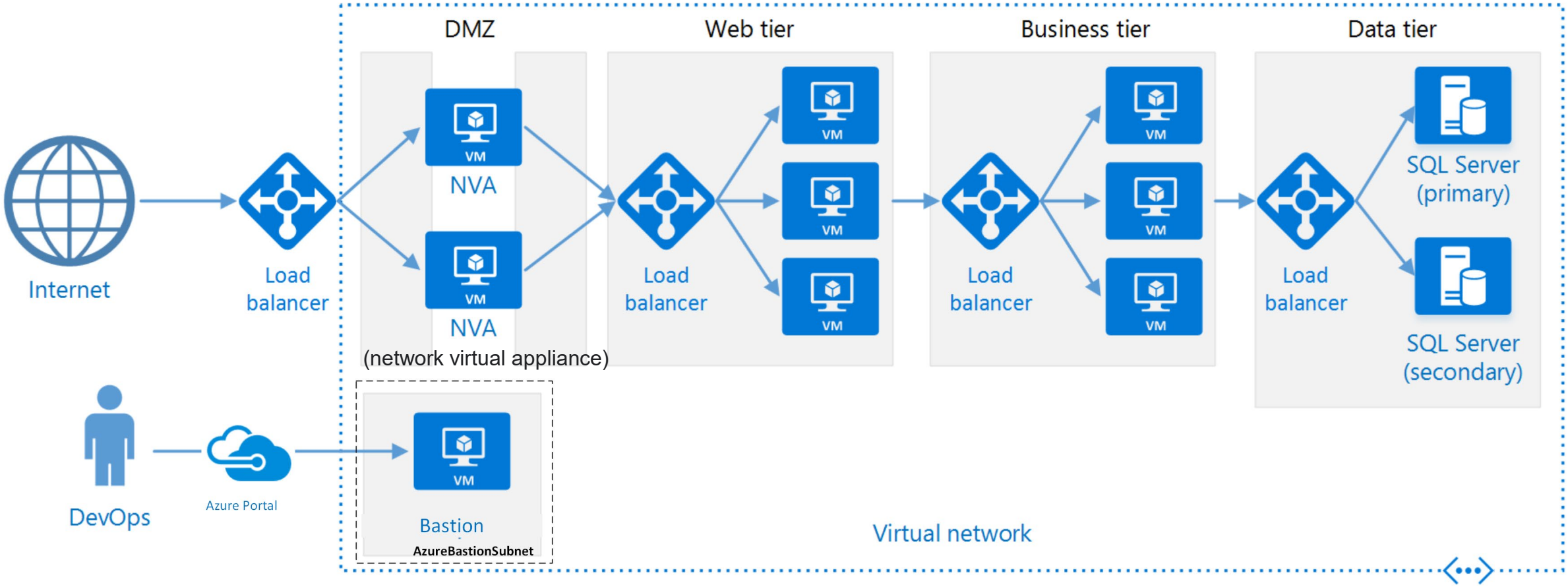


Ulster University

N-Tier

Benefits	Challenges
Portability between cloud and on-premises, and between cloud platforms	It's easy to end up with a middle tier that just does CRUD operations on the database, adding extra latency without doing any useful work
Less learning curve for most developers	Monolithic design prevents independent deployment of features
Natural evolution from the traditional application model	Managing an IaaS application is more work than an application that uses only managed services
Open to heterogeneous environment (Windows/Linux)	It can be difficult to manage network security in a large system

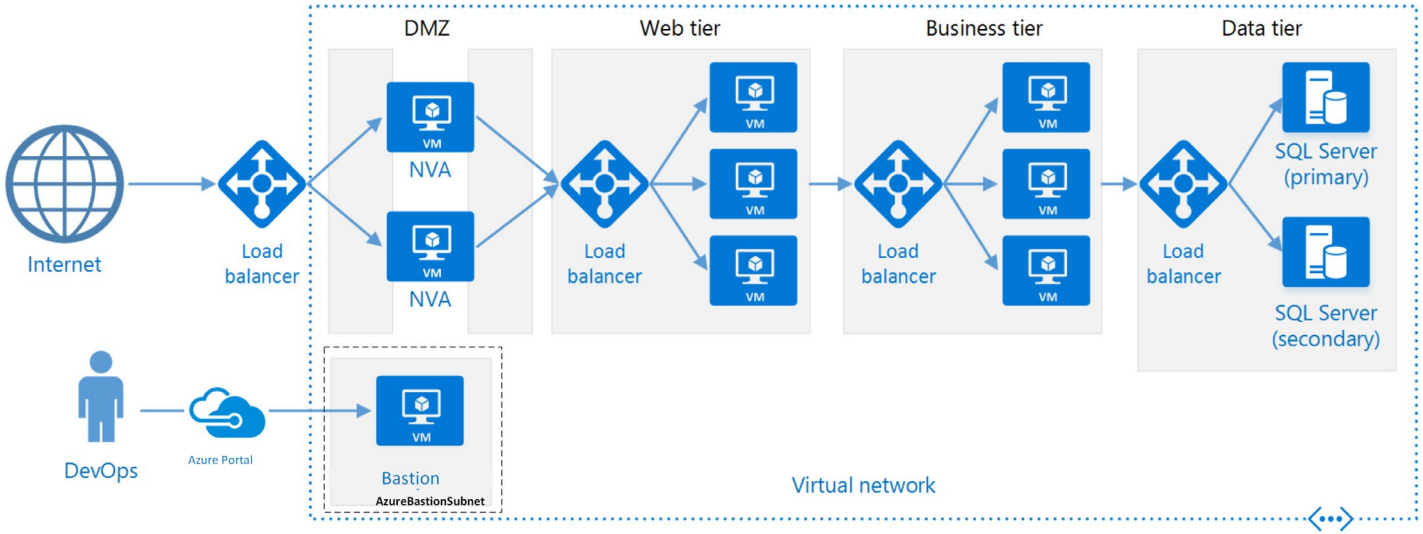
N-Tier on a VM set



N-Tier

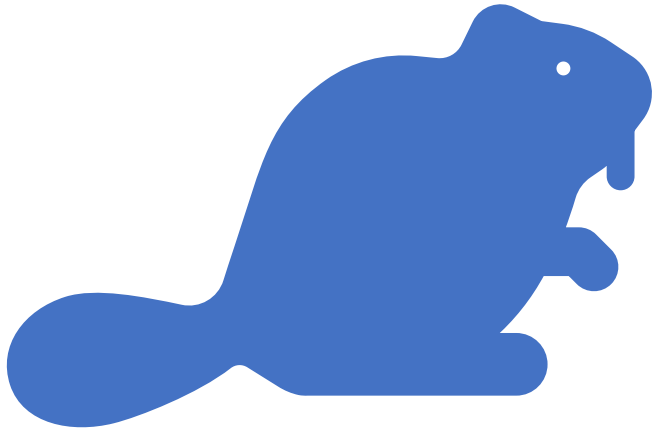
In this scenario:

- Each tier consists of two or more VMs, placed in an availability set or virtual machine scale set.
- Multiple VMs provide resiliency in case one VM fails.
- Load balancers are used to distribute requests across the VMs in a tier.
- A tier can be scaled horizontally by adding more VMs to the pool.

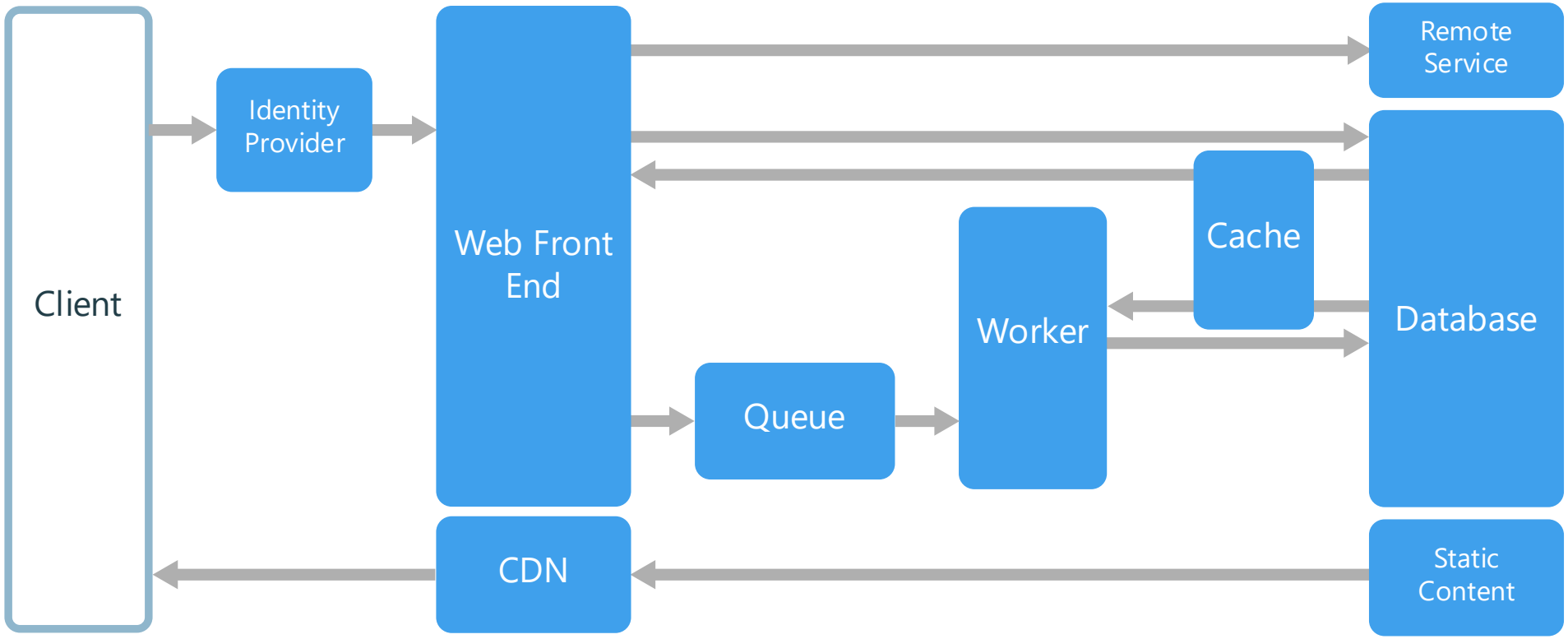


Ulster University

Web-Queue-Worker



Ulster University

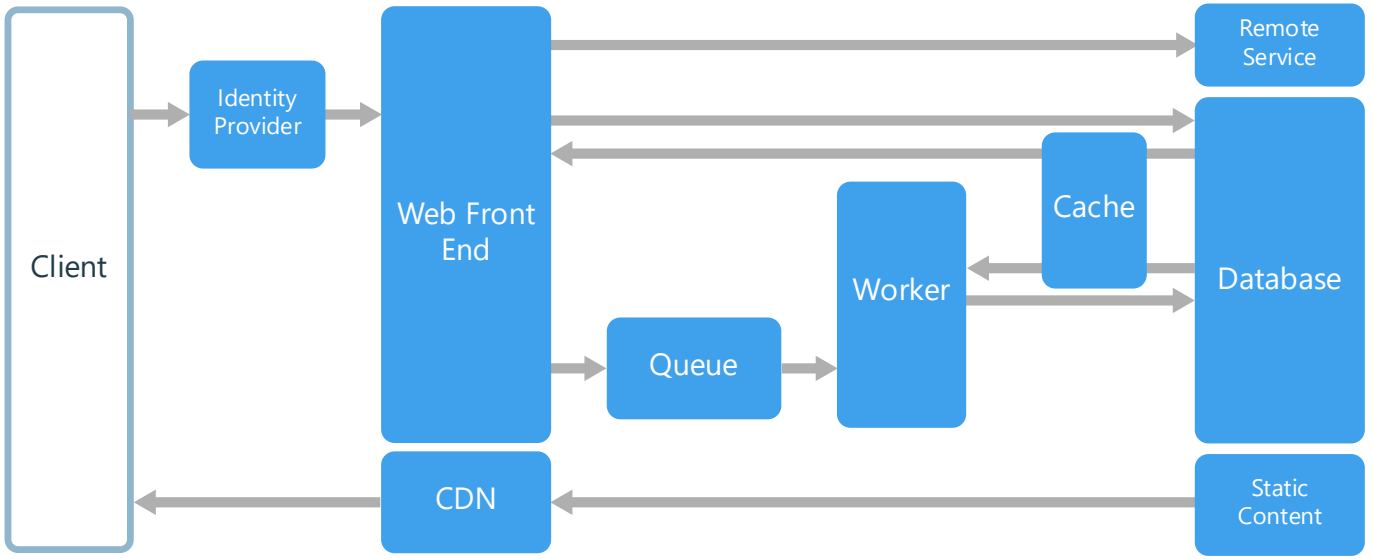


Web-Queue-Worker

Web-Queue-Worker

The core components of this architecture are:

- A **web front end** that serves client requests
- A **worker** that performs resource-intensive tasks, long-running workflows, or batch jobs.
- A **Message Queue** to enable the web front end to communicate with the worker.



Ulster University

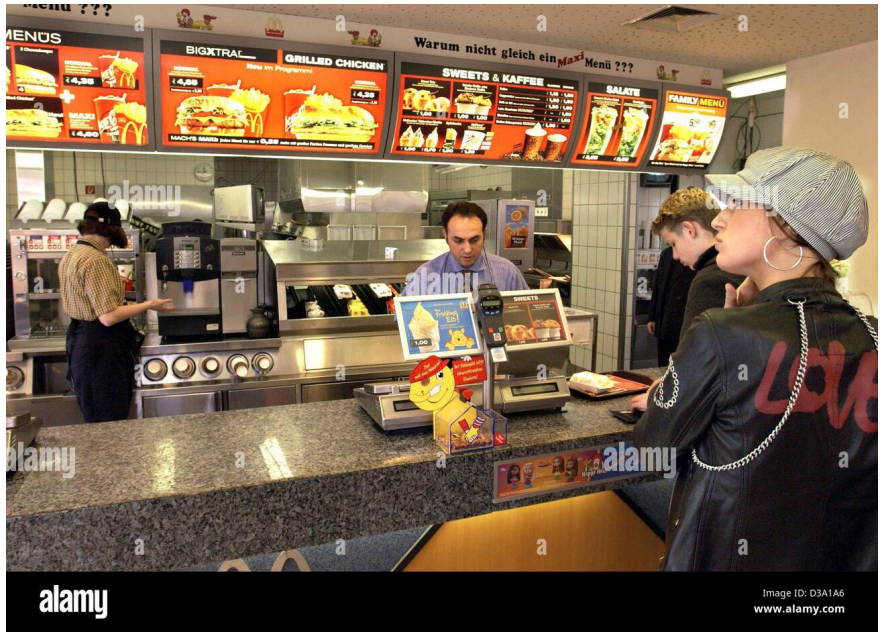
Web-Queue-Worker

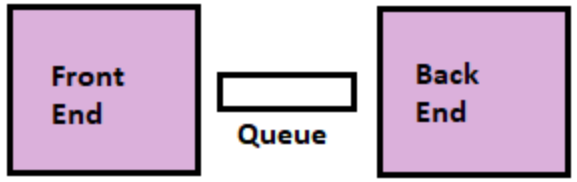
*For a purely **PaaS** solution, consider a Web-Queue-Worker architecture.*

In this style, the application has a web front end that handles HTTP requests and a back-end worker that performs CPU-intensive tasks or long-running operations.

The front end communicates to the worker through an asynchronous message queue.

Web-queue-worker is suitable for relatively simple domains with some resource-intensive tasks.





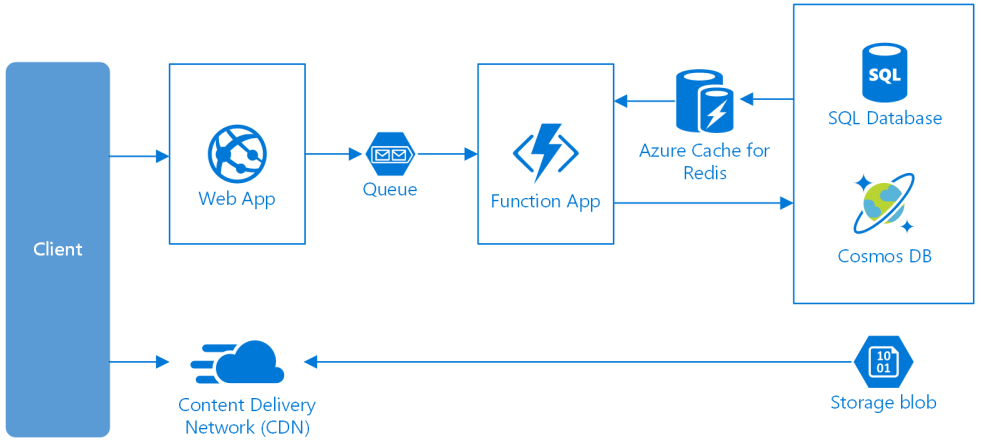
Web-Queue-Worker

Web-Queue-Worker

Like N-tier, the architecture is easy to understand.

The use of managed services simplifies deployment and operations. But with complex domains, it can be hard to manage dependencies.

The front end and the worker can easily become large, monolithic components that are hard to maintain and update. As with N-tier, this can reduce the frequency of updates and limit innovation/agility.

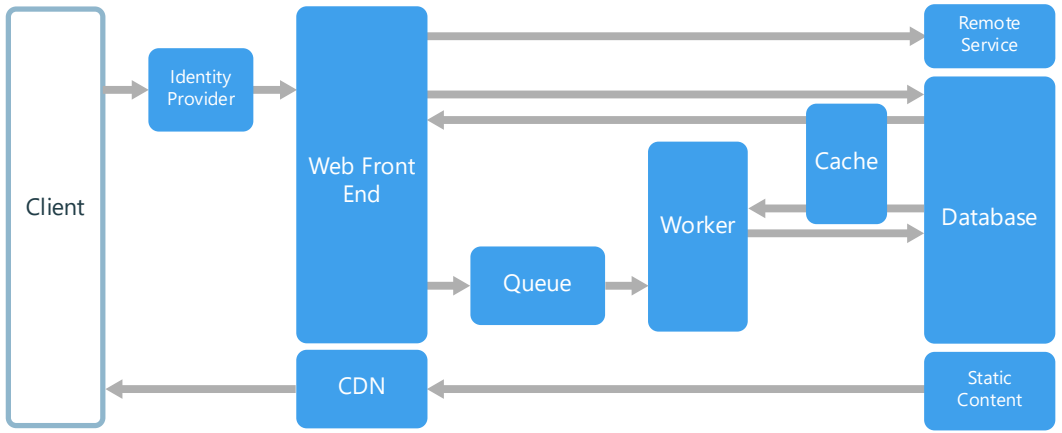


Ulster University

Web-Queue-Worker

Other components that are commonly incorporated into this architecture include:

- One or more **databases**.
- A **cache** to store values from the database for quick reads.
- A **Content Delivery Network (CDN)** to serve static content
- **Remote services**, such as email or SMS service.
- **Identity provider** for authentication.



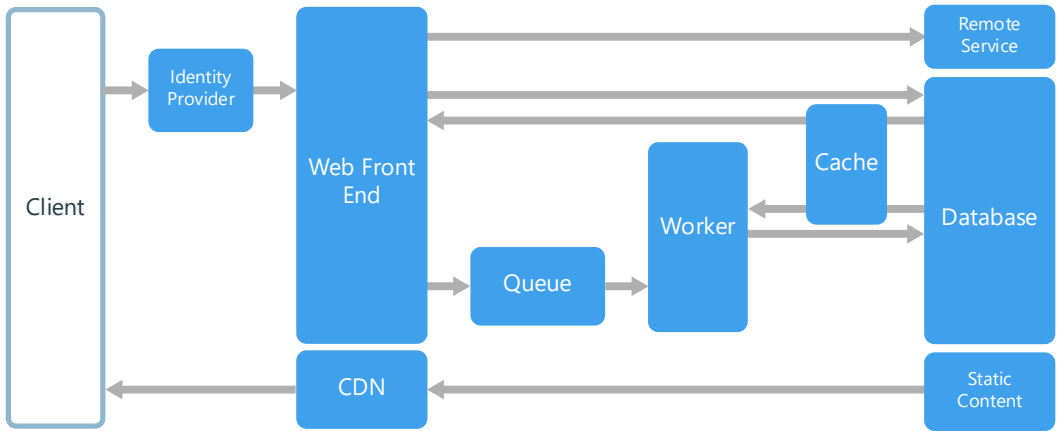
Web-Queue-Worker

The web and worker are both stateless; session state can be stored in a distributed cache.

Any long-running work is done asynchronously by the worker. The worker can be triggered by messages on the queue, or run on a schedule for batch processing.

The worker is an *optional component*. If there are no long-running operations, the worker can be omitted.

The front end might consist of a web API. On the client side, the web API can be consumed by a single-page application that makes AJAX calls, or by a native client application.



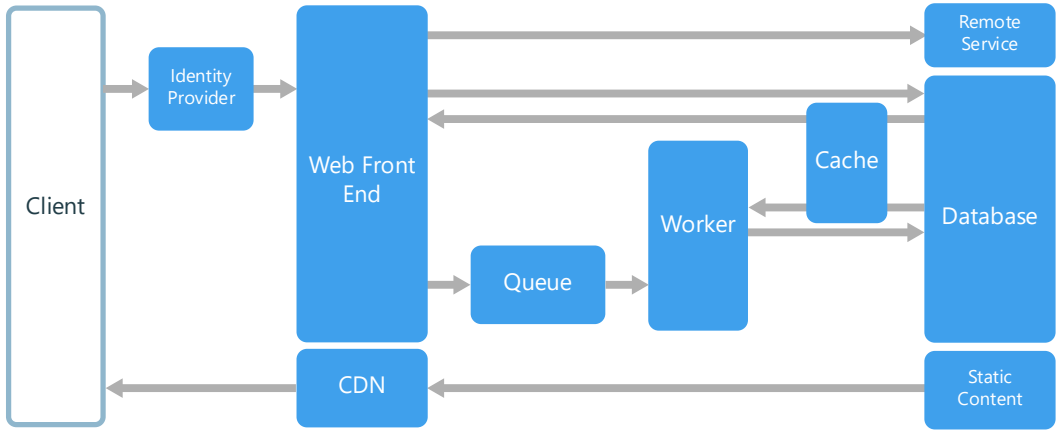
Web-Queue-Worker

When to use this architecture

The Web-Queue-Worker architecture is typically implemented using managed compute services, either Azure App Service or Azure Cloud Services.

Consider this architecture style for:

- Applications with a **relatively simple domain**.
- Applications with some **long-running workflows** or **batch operations**.
- When you **want to use managed services**, rather than infrastructure as a service (IaaS).

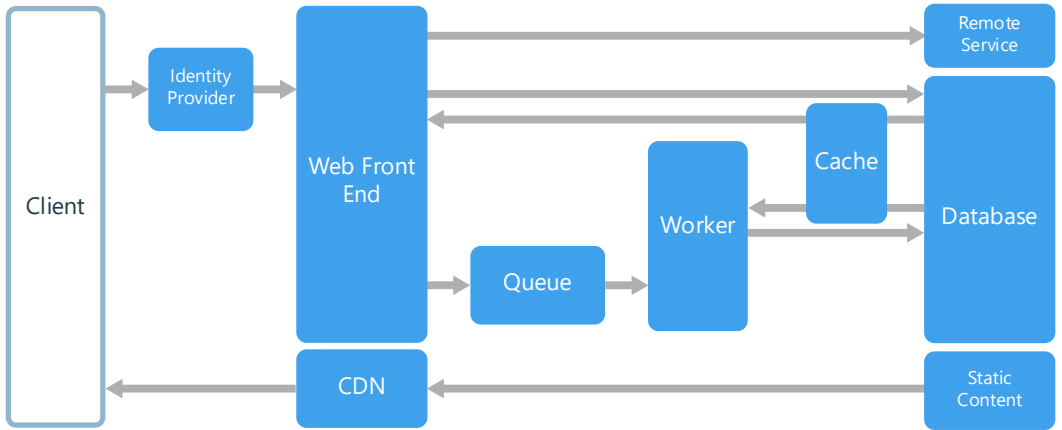


Web-Queue-Worker

Benefits

This architecture has many benefits, including:

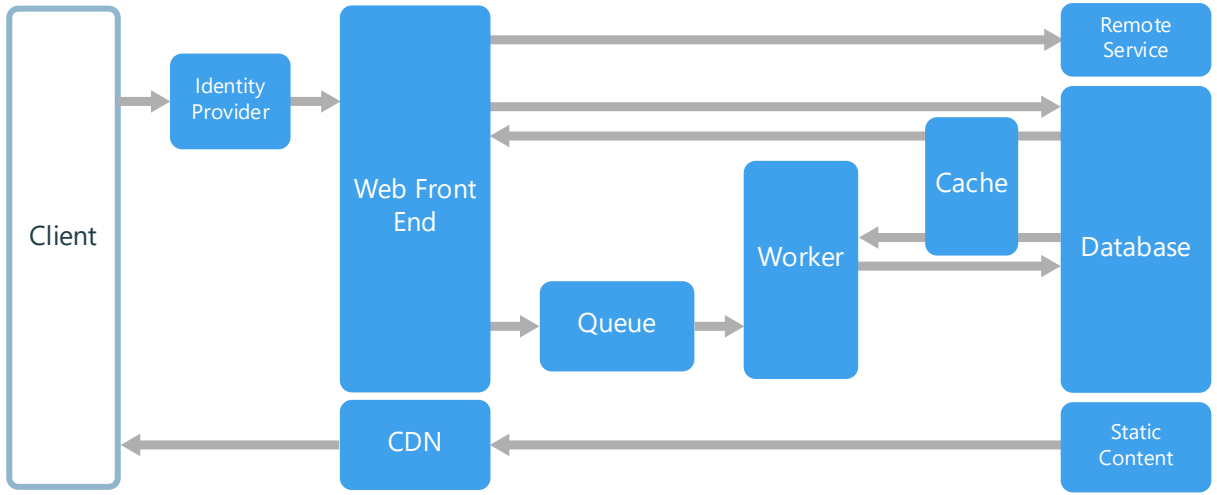
- Relatively simple architecture that is easy to understand.
- Easy to deploy and manage.
- Clear separation of concerns.
- The front end is decoupled from the worker using asynchronous messaging.
- The front end and the worker can be scaled independently.



Challenges

Without careful design, the front end* and the worker can become large, monolithic components that are difficult to maintain and update.

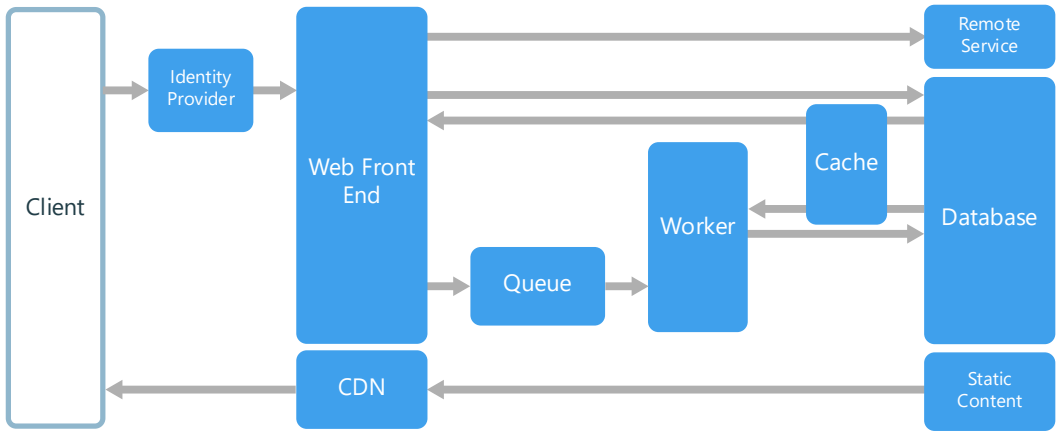
There may be hidden dependencies, if the front end and worker share data schemas or code modules.

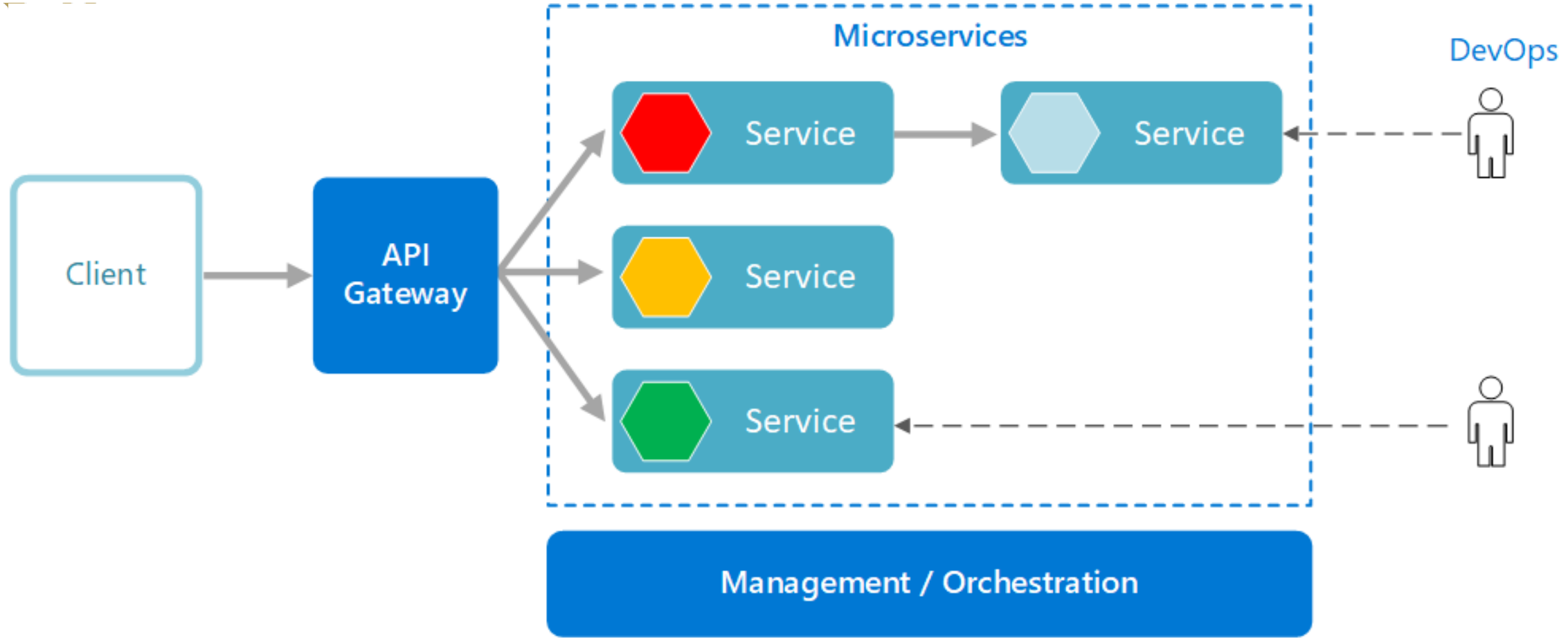


Web-Queue-Worker

Best practices

- Expose a **well-designed API** to the client.
- **Autoscale** to handle changes in load.
- **Cache** semi-static data. Use a CDN to host static content.
- Use polyglot persistence when appropriate; **use the best data store for the job**.
- **Partition data to improve scalability**, reduce contention, and optimize performance.



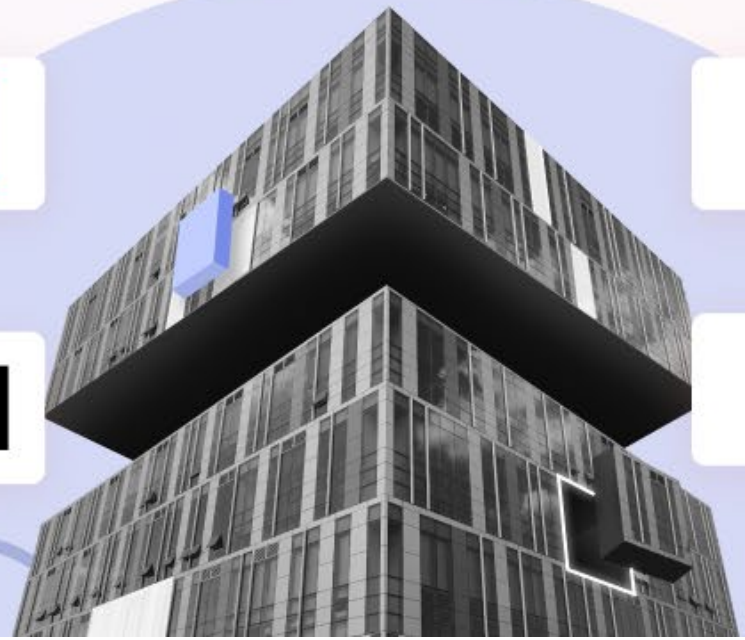


Microservices

Ulster University

SIMFORM

Microservices Example



Etsy

CapitalOne

LEGO

NIKE

Goldman Sachs

Uber

GROUPON

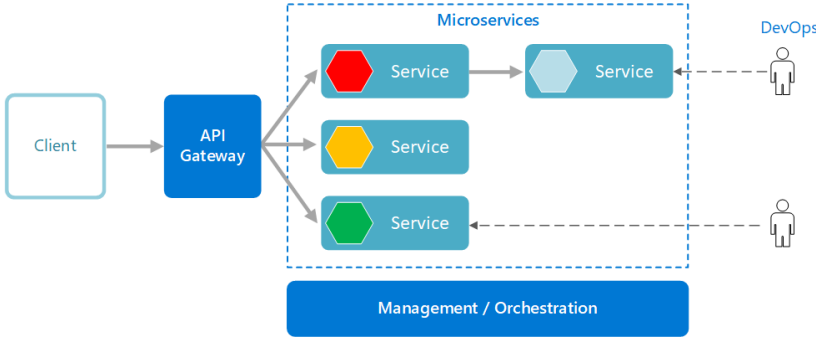
GILT

lyft

Overview

- Microservices are small, independent, and loosely coupled.
  A single small team of developers can write and maintain a service.
- Each service is a separate codebase, which can be managed by a small development team.
- Services can be deployed independently. A team can update an existing service without rebuilding and redeploying the entire application.

[Flightradar24: Live Flight Tracker - Real-Time Flight Tracker Map]



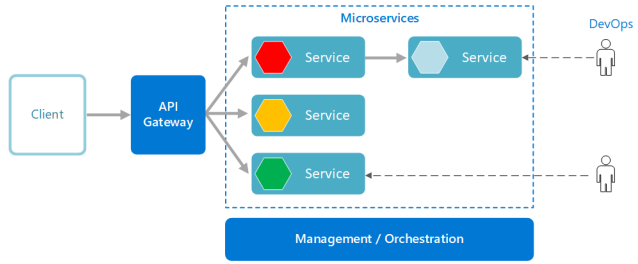
Overview

- Services are responsible for persisting their own data or external state.

  This differs from the traditional model, where a separate data layer handles data persistence.

- Services communicate with each other by using well-defined APIs.
  Internal implementation details of each service are hidden from other services.

- Supports polyglot programming.
  For example, services don't need to share the same technology stack, libraries, or frameworks.



Microservice Architecture

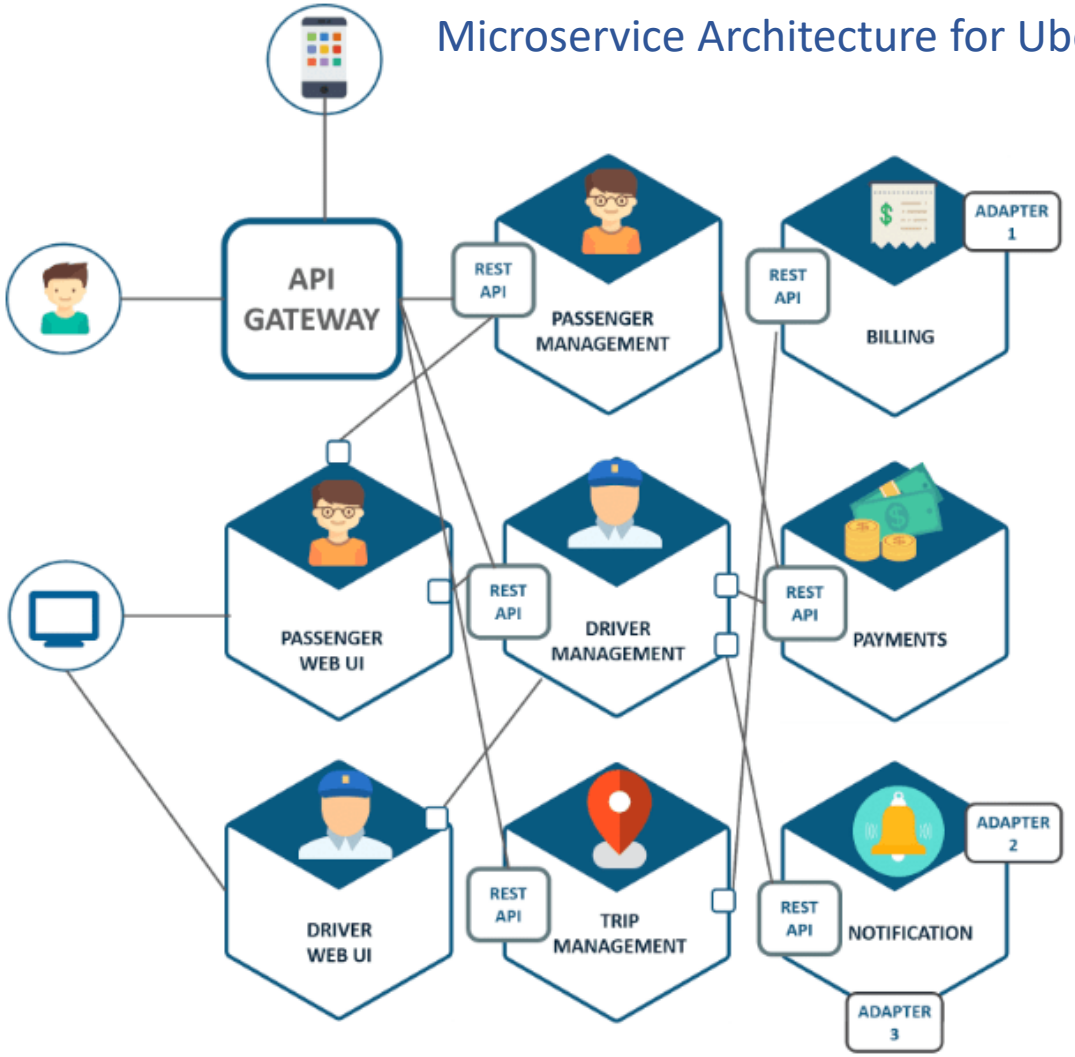
Microservice Architecture for Uber

If your application has a more complex domain, consider moving to a Microservices architecture.

A microservices application is composed of many small, independent services.

Each service implements a single business capability. Services are loosely coupled, communicating through API contracts.

Each service can be built by a small, focused development team.



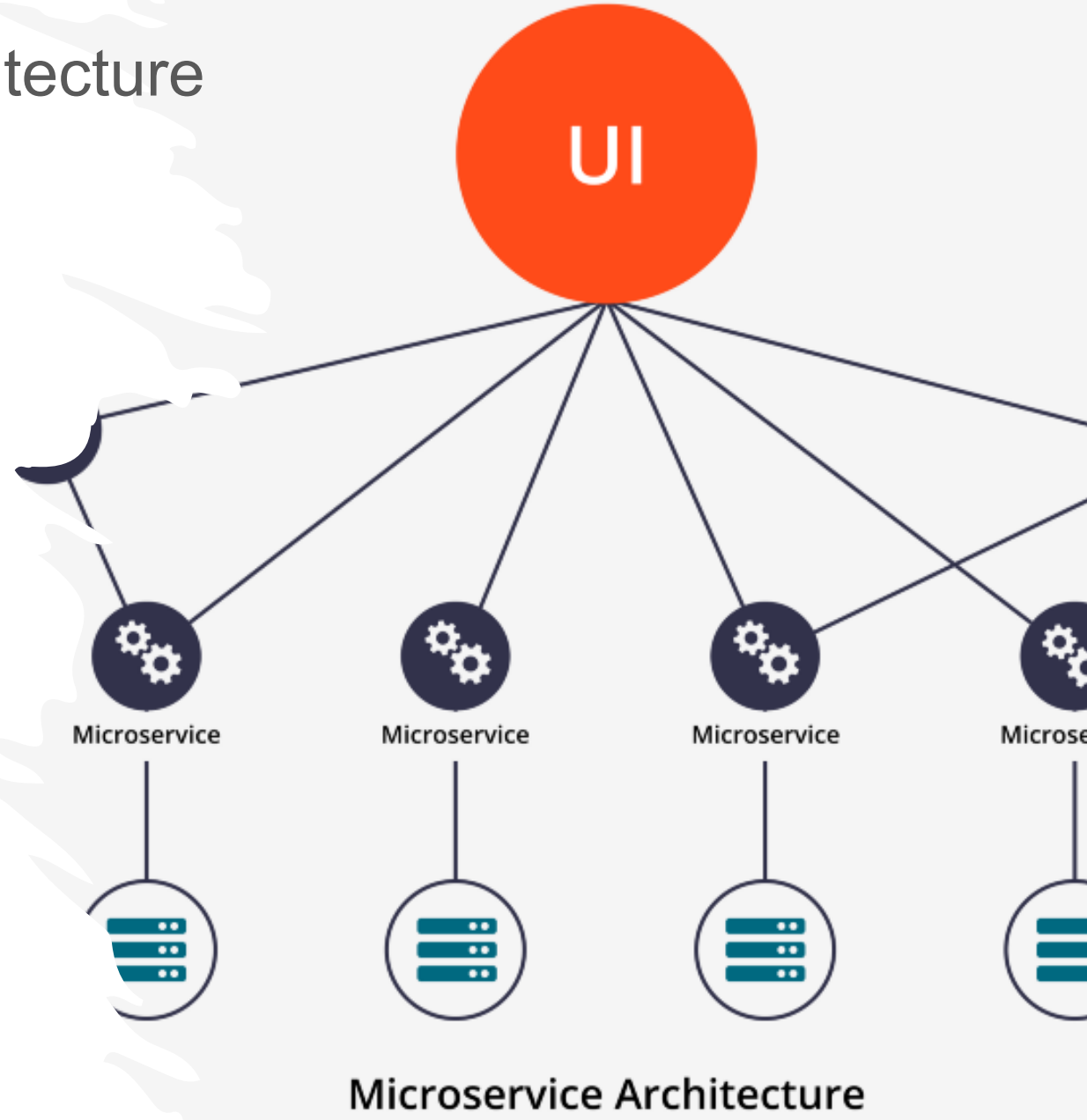
Microservice Architecture

Individual services can be deployed without a lot of coordination between teams, which encourages frequent updates.

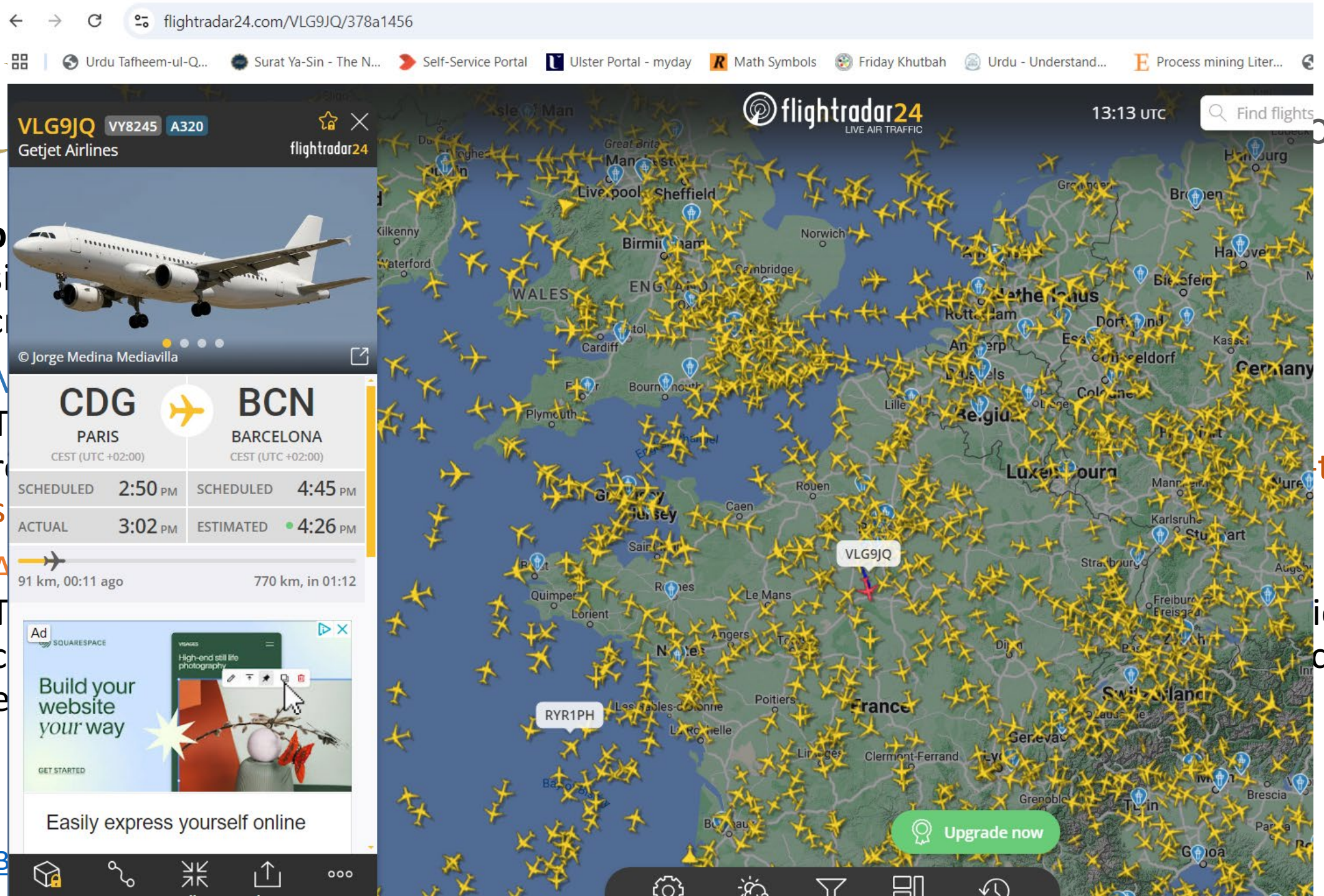
A microservice architecture is more complex to build and manage than either N-tier or web-queue-worker.

It requires a mature development and DevOps culture.

Done right, this style can lead to higher release velocity, faster innovation, and a more resilient architecture.



Microservice Architecture



Sup
Besi
micr

- M
  T
  r
  s
- A
  T
  c
  e

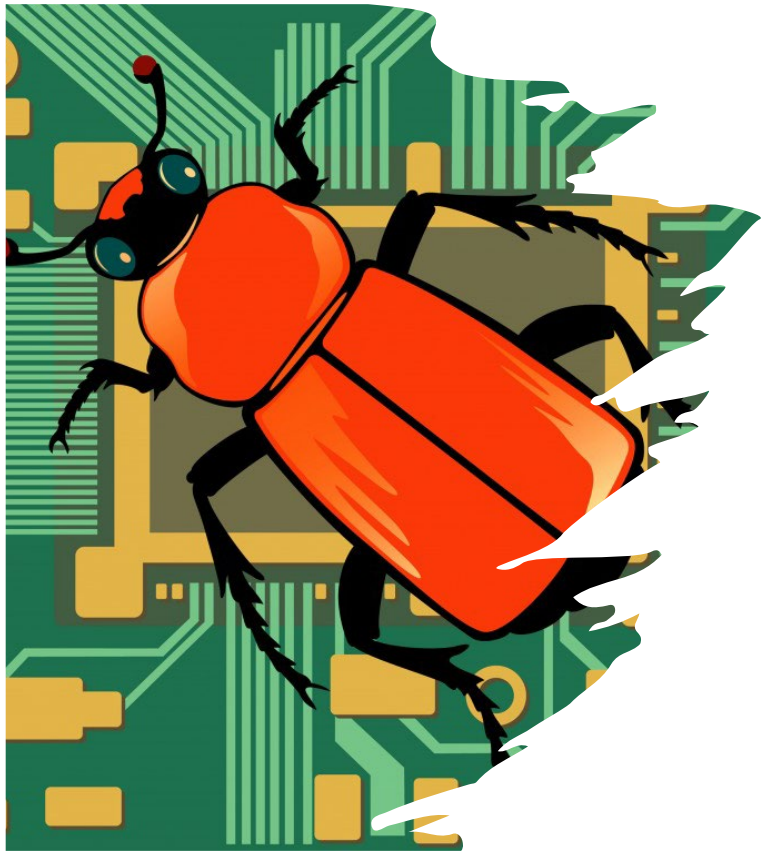
Belfast, B

ces

-the-

ients

ck



Benefits of Microservices

Agility

Because microservices are deployed independently, it's easier to manage bug fixes and feature releases.

You can update a service without redeploying the entire application, and roll back an update if something goes wrong.

In many traditional applications, if a bug is found in one part of the application, it can block the entire release process.

New features may be held up waiting for a bug fix to be integrated, tested, and published.

Benefits of Microservices

Mix of technologies.

Teams can pick the technology that best fits their service, using a mix of technology stacks as appropriate.

Fault isolation.

If an individual microservice becomes unavailable, it won't disrupt the entire application, as long as any upstream microservices are designed to handle faults correctly (for example, by implementing circuit breaking).

Benefits of Microservices

Small Code Base

In a monolithic application, there is a tendency over time for code dependencies to become tangled.

Adding a new feature requires touching code in a lot of places.

By not sharing code or data stores, a microservices architecture minimizes dependencies, and that makes it easier to add new features.

Ulster University

Microservices

Challenges

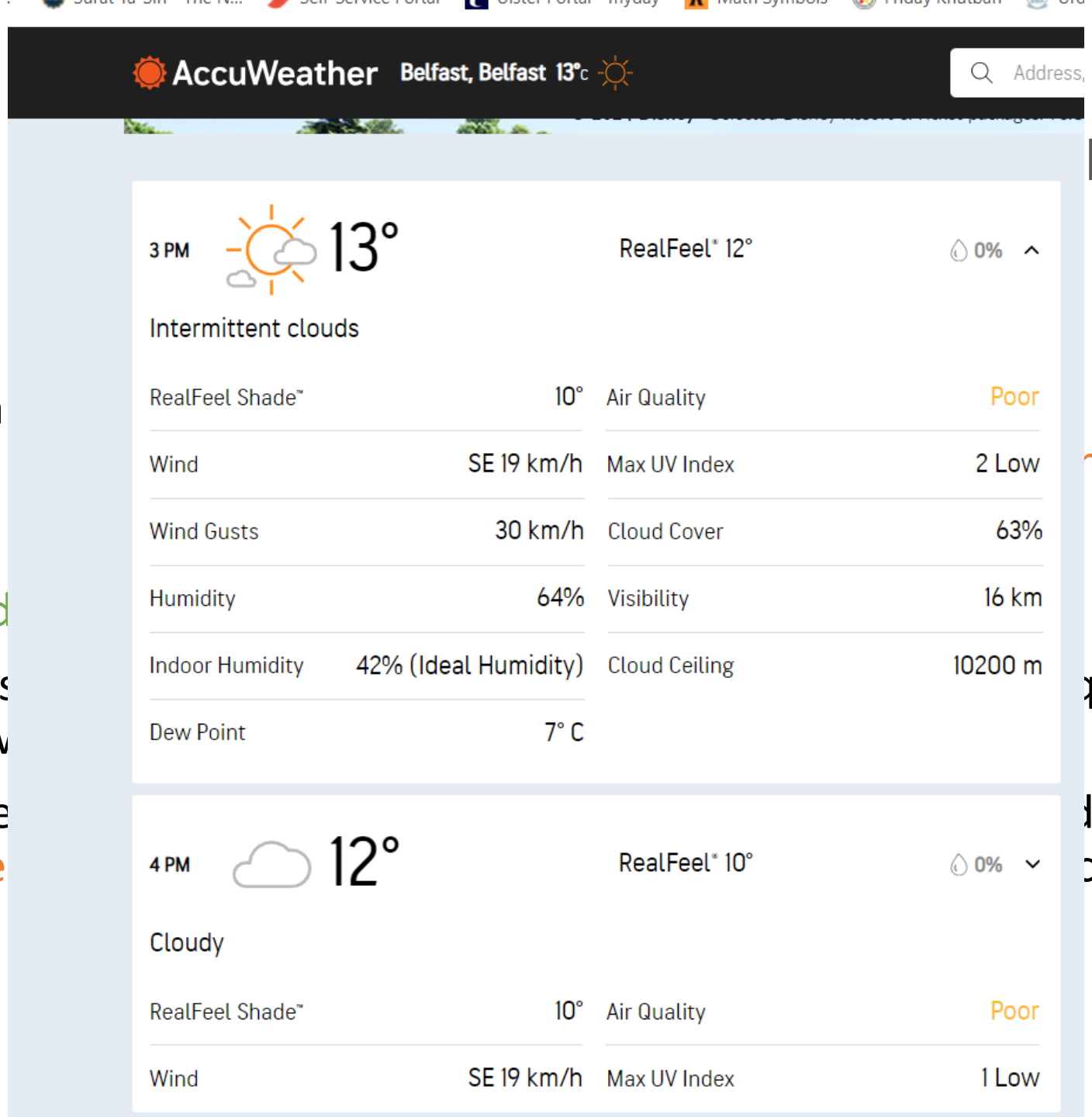
Complexity

A microservices a monolithic application. Each more complex.

Development and

Writing a small s quires a different approach than a v

Existing tools are dencies. It is also challenging to te cation is evolving quickly.



Challenges

Lack of governance

The decentralized approach to building microservices has advantages, but it can also lead to problems.

You may end up with so many different languages and frameworks that the application becomes hard to maintain.

It may be useful to put some project-wide standards in place, without overly restricting teams' flexibility. This especially applies to cross-cutting functionality such as logging.

Challenges

Data integrity

With each microservice responsible for its own data persistence. As a result, data consistency can be a challenge. Embrace eventual consistency where possible.

Management

To be successful with microservices requires a mature DevOps culture. Correlated logging across services can be challenging. Typically, logging must correlate multiple service calls for a single user operation.

Challenges

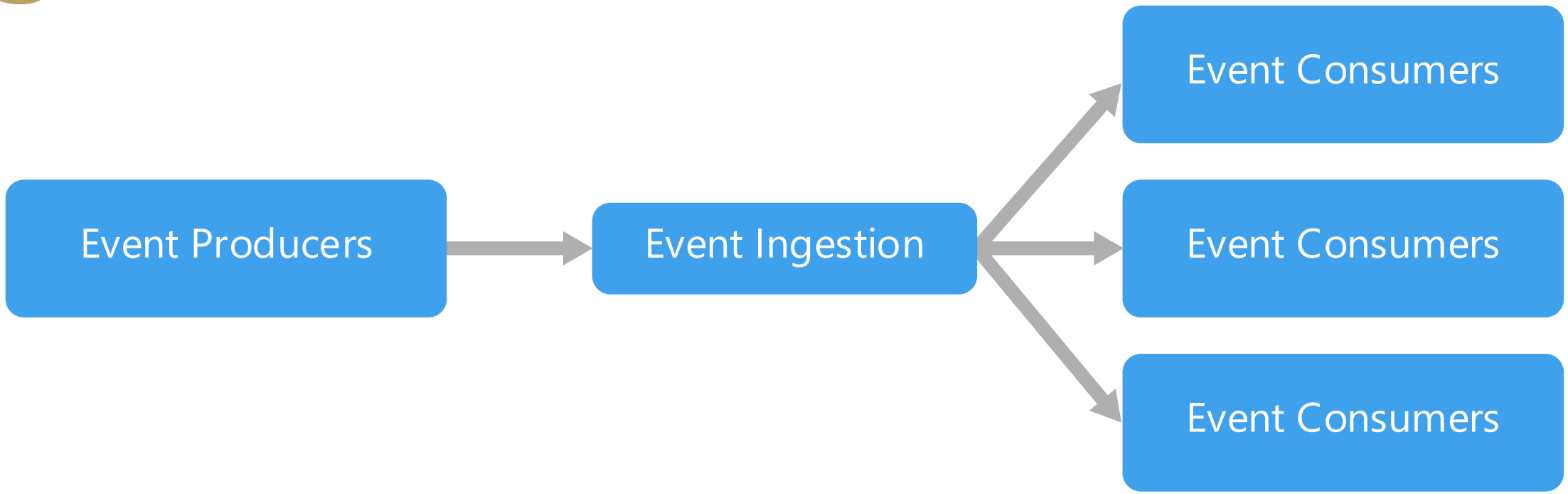
Versioning

Updates to a service must not break services that depend on it. Multiple services could be updated at any given time, so without careful design, you might have problems with backward or forward compatibility.

Skill set

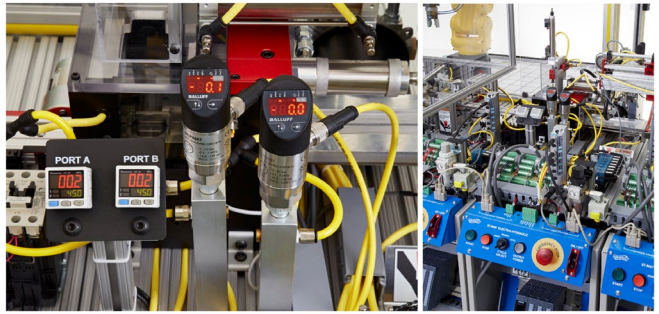
Microservices are highly distributed systems. Carefully evaluate whether the team has the skills and experience to be successful.


Event Driven Architecture



Event Driven Architecture

Ulster University



Event Producers → Event Ingestion → Event Consumers, Event Consumers, Event Consumers

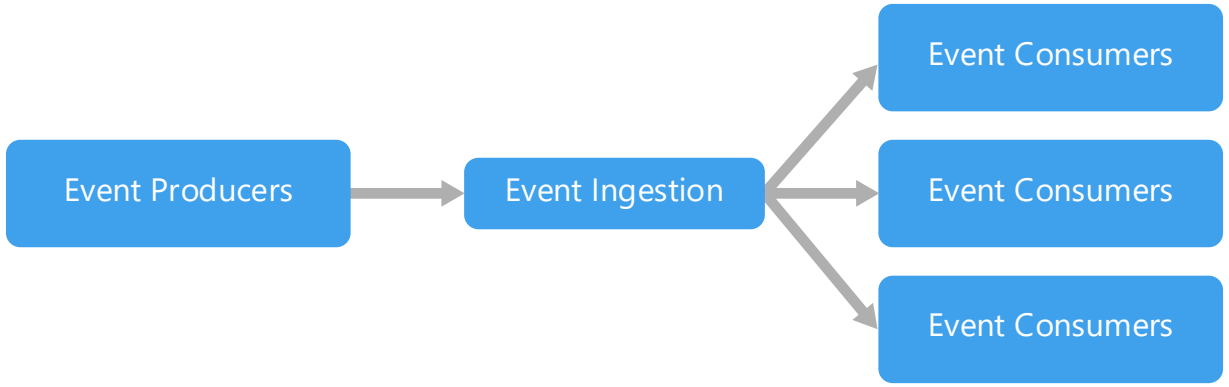
Event Driven Architecture

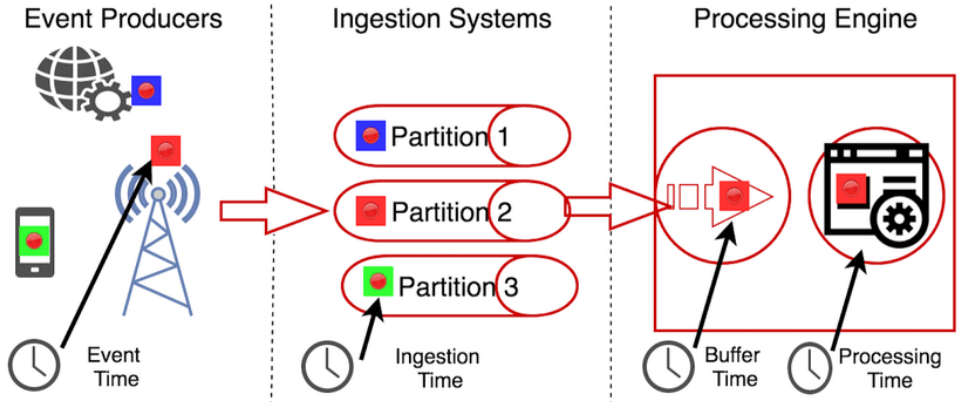
An *event-driven architecture* consists of **event producers** that generate a **stream** of **events**, and **event consumers** that listen for the events.

Event-Driven Architecture

Events are delivered in near real time, so consumers can respond immediately to events as they occur. These have some characteristics:

- Producers are **decoupled** from consumers.
- A producer typically **doesn't know which consumers** are listening.
- Consumers are also **decoupled** from each other.
- Every consumer sees **all** of the events.
- In some systems, such as IoT, events must be ingested at very high volumes.



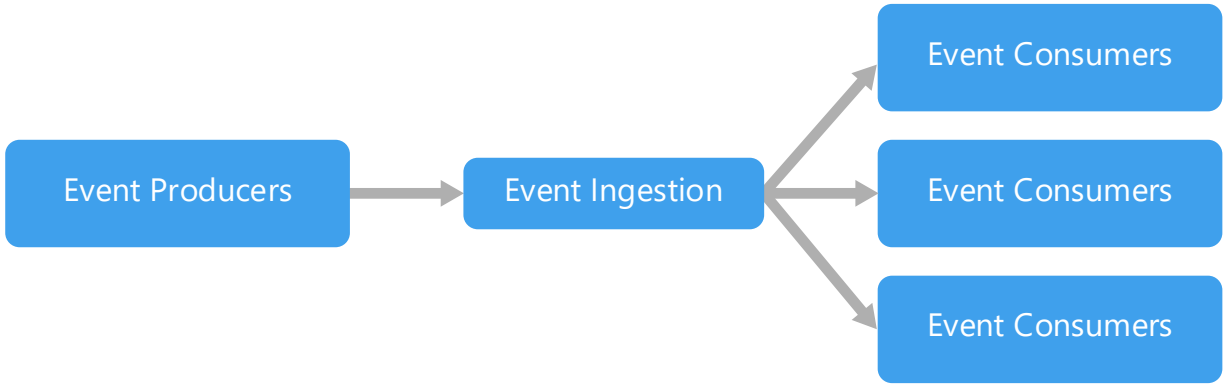


Event-Driven Architecture

An event driven architecture can use a **pub/sub** model or an **event stream** model.

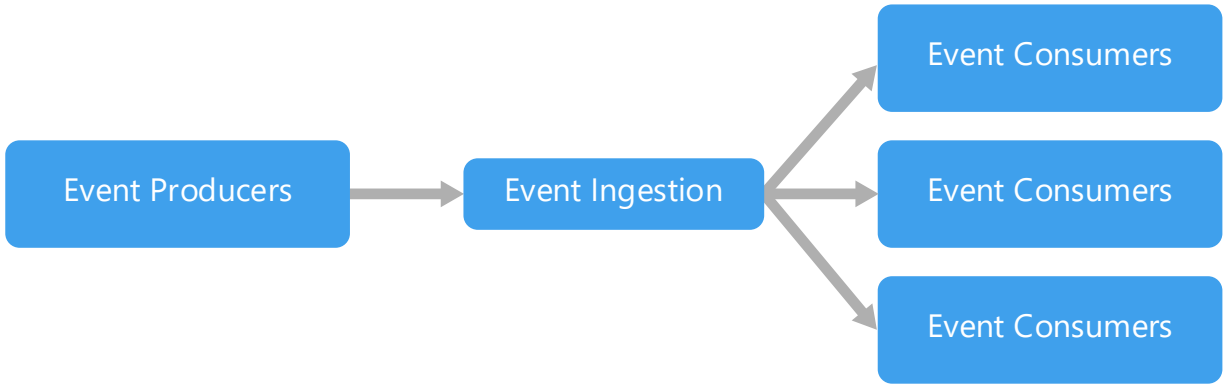
Approach 01- Pub/sub

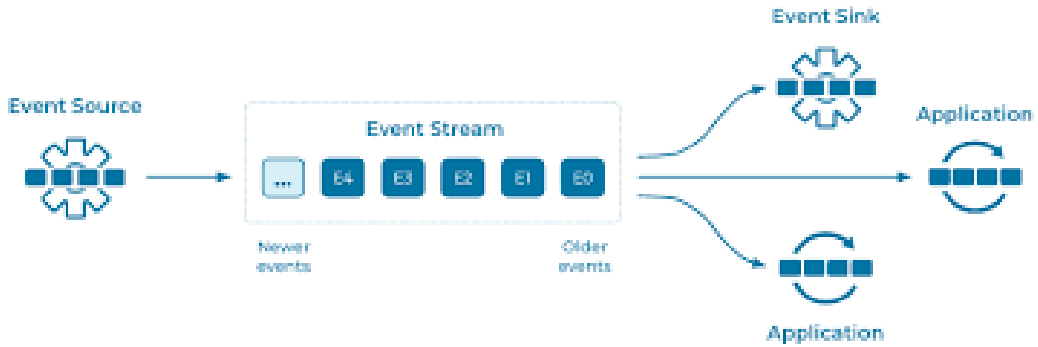
- The messaging infrastructure keeps track of subscriptions.
- When an event is published, it sends the event to each subscriber.
- After an event is received, it cannot be replayed
- New subscribers do not see the event.



Approach 02- Event streaming

- Events are written to a log.
- Events are strictly ordered and durable.
- Clients don't subscribe to a stream, instead they can read from any part of a stream.
- The client is responsible for advancing its position in the stream. That means a client can join at any time, and can replay events.

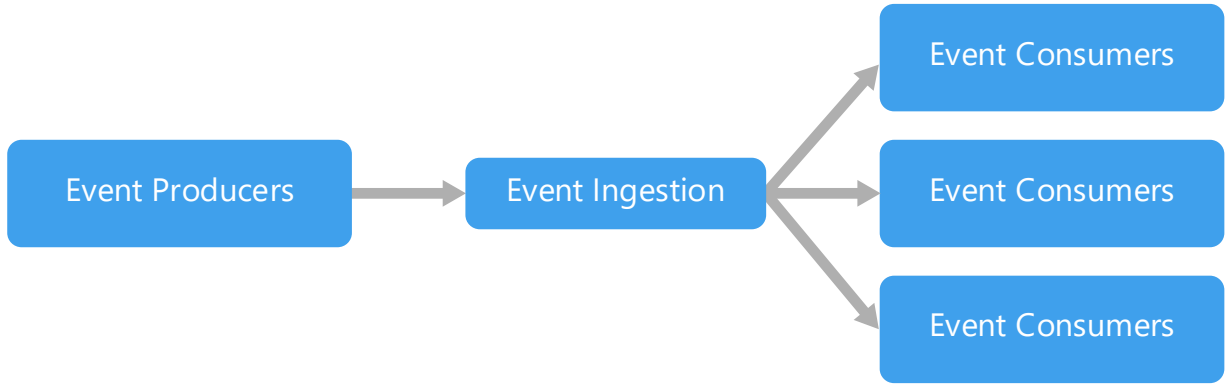




Event-Driven Architecture

When to use this architecture

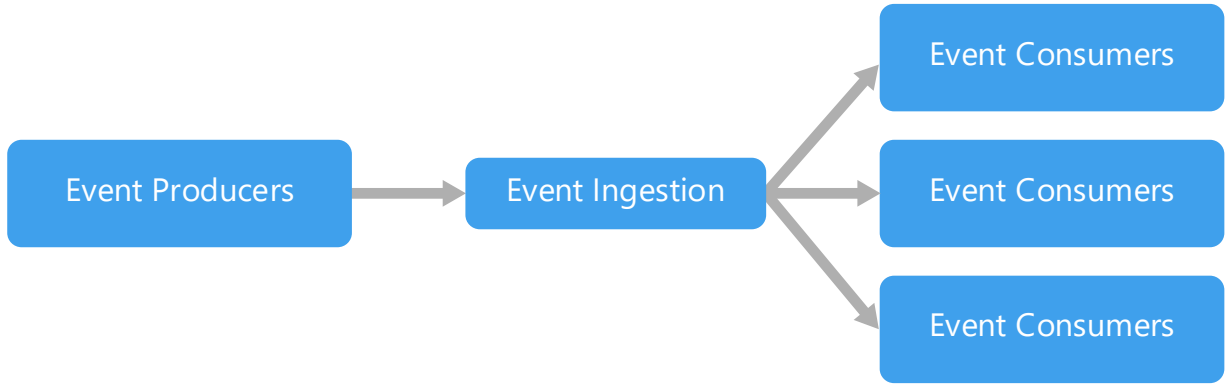
- Multiple subsystems must process the same events.
- Real-time processing with minimum time lag.
- Complex event processing, such as pattern matching or aggregation over time windows.
- High volume and high velocity of data, such as IoT.



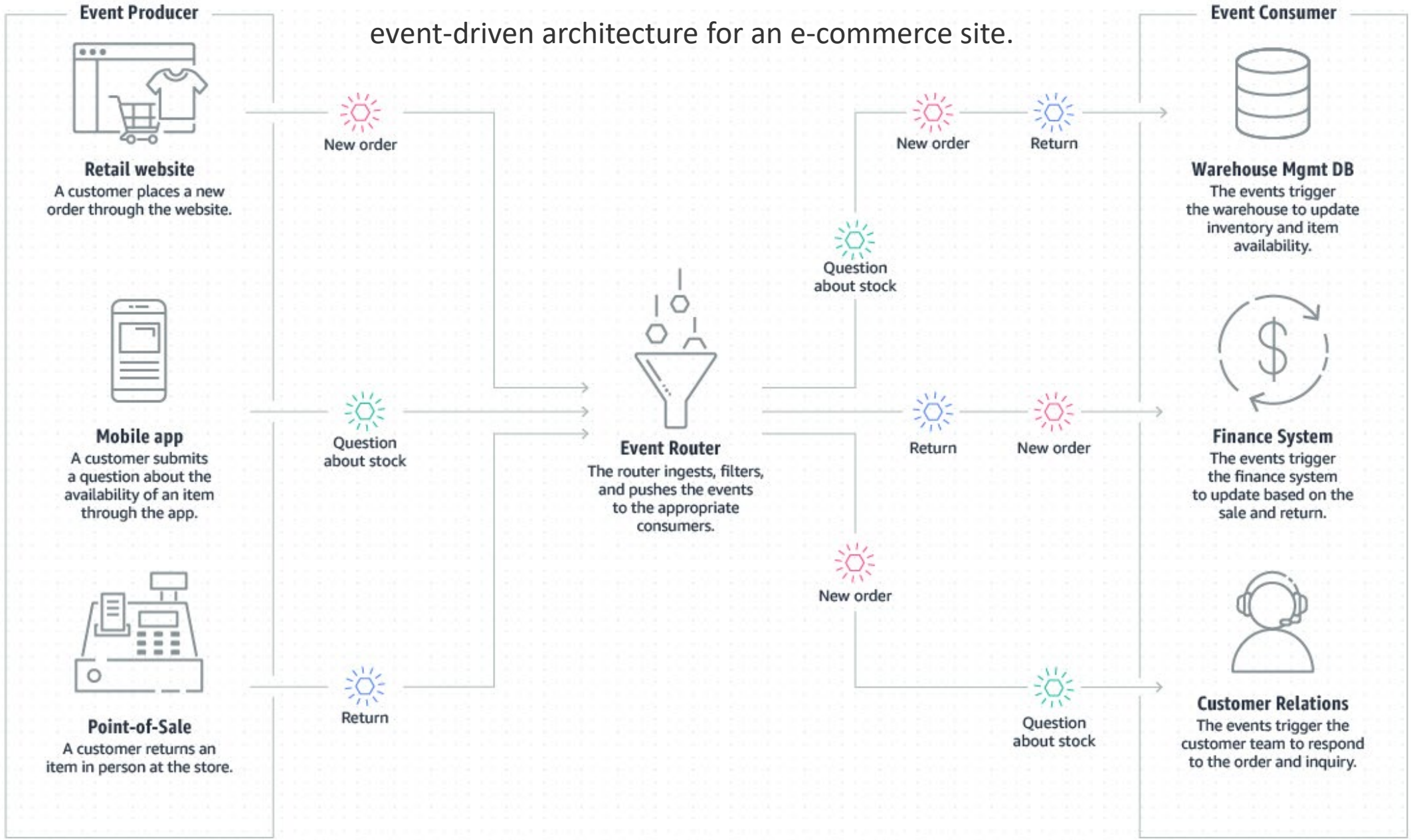
Event-Driven Architecture

Benefits

- Producers and consumers are decoupled.
- No point-to-point integrations. It's easy to add new consumers to the system.
- Consumers can respond to events immediately as they arrive.
- Highly scalable and distributed.
- Subsystems have independent views of the event stream.



event-driven architecture for an e-commerce site.



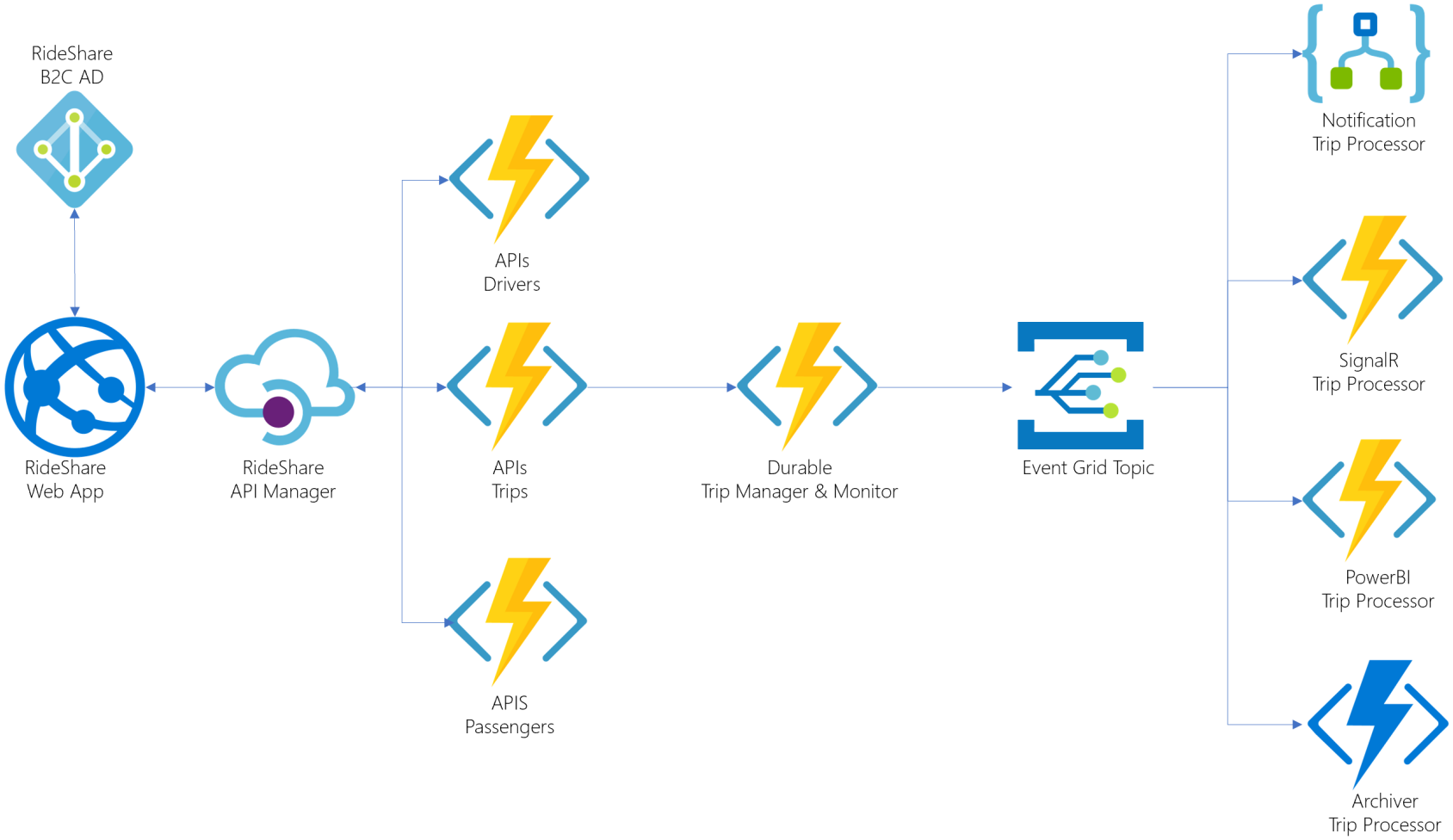
Ulster University

Architectural Comparison

Architecture style	Dependency management	Domain type
N-tier	Horizontal tiers divided by subnet	Traditional business domain. Frequency of updates is low.
Web-Queue-Worker	Front and backend jobs, decoupled by async messaging.	Relatively simple domain with some resource intensive tasks.
Microservices	Vertically (functionally) decomposed services that call each other through APIs.	Complicated domain. Frequent updates
Event-driven architecture.	Producer/consumer. Independent view per sub-system.	IoT and real-time systems

Ulster University

Serverless Architecture



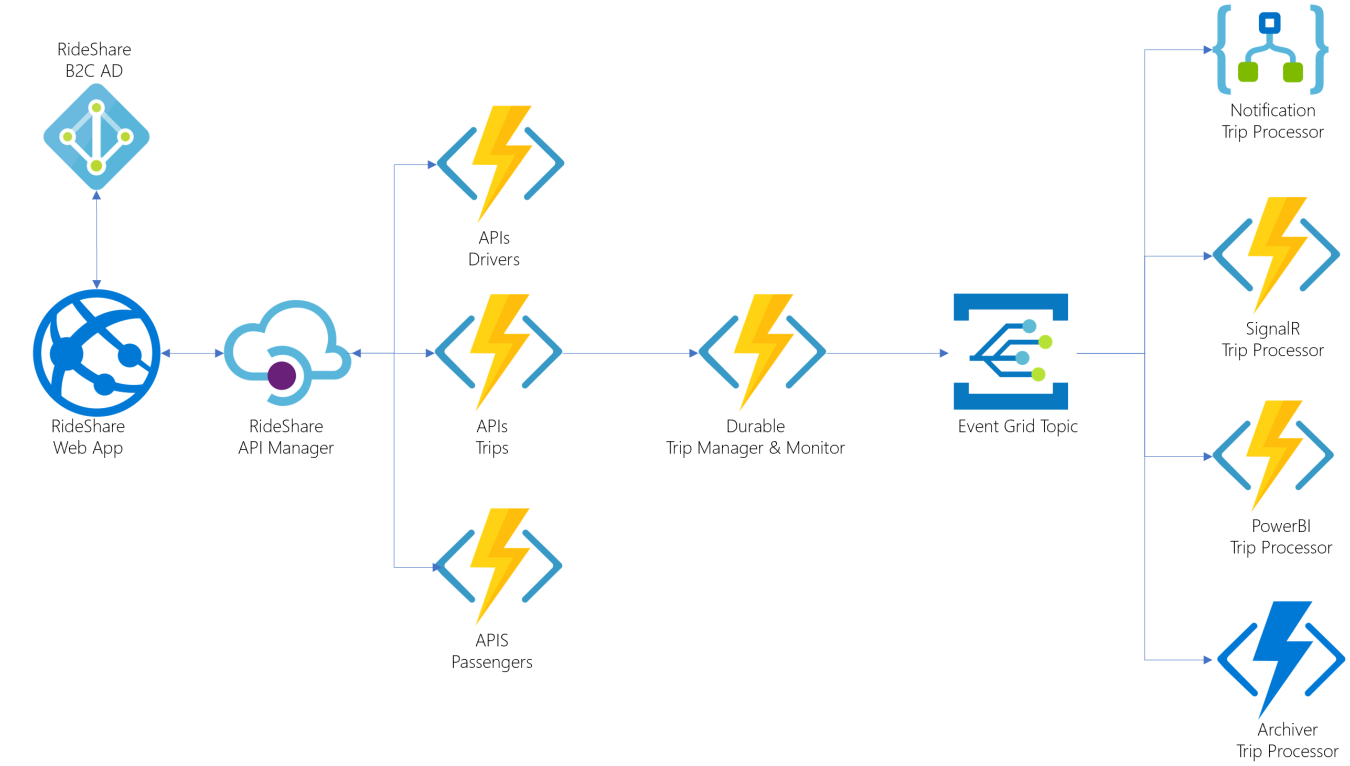
Serverless Architecture

What Really is Serverless?

- **Definition:** Cloud execution model with provider-managed servers
- Focus on code, not infrastructure

- **Key Characteristics:**
  - No Server Management
  - Event-Driven Execution
  - Auto, Granular Scaling
  - Pay-Per-Use Billing

- **Analogy:** Like an electrical grid—you use what you need



Ulster University

Serverless Architecture

Azure's Serverless Toolbox: More Than Just Functions

Service	Role	Key Purpose
Azure Functions	Event-driven Compute	Executes your code logic
Azure Logic Apps	Workflow Automation	Orchestrates processes with a visual designer
Azure Event Grid	Event Routing	Connects event sources to handlers (the "nervous system")
Azure Service Bus	Reliable Messaging	Decouples services with message queues
Azure API Management	API Gateway	Manages, secures, and throttles API calls

Key Idea: Build applications by composing these serverless services.

Ulster University

Serverless Architecture

Azure Functions: Your Code in a Serverless Container

Content:
The primary FaaS (Function-as-a-Service) offering on Azure.

How it Works:

- Write a function – a single method in your preferred language.
- Define a **Trigger** (what causes the function to run).
- Use **Bindings** to connect to other services (input/output).

Conceptual Flow:
[Trigger] -> [Your Function Code] -> [Output Binding]

Common Triggers:

- HTTP (build a web API)
- Timer (run on a schedule)
- Azure Queue Storage (process a message)
- Azure Event Grid (react to an event)
- Cosmos DB (react to data changes)

1. Event Processing & Automation

- *Example:* Resize an image on upload.
- (Blob Trigger) -> (Resize Function) -> (Save to new container)

2. RESTful APIs & Backends

- *Example:* Mobile app backend.
- (HTTP Trigger) -> (Function logic) -> (Returns JSON)

3. Data Processing Pipelines

- *Example:* Process IoT device data.
- (Event Hub Trigger) -> (Process Function) -> (Output to Cosmos DB)

4. Scheduled Tasks (Cron Jobs)

- *Example:* Daily database cleanup.
- (Timer Trigger) -> (Cleanup Function)

Serverless Architecture

Aspect	Serverless (Functions)	Microservices (AKS)	N-Tier (VMs)
Management	**Very Low**	Medium	High
Scaling	**Automatic & Instant**	Manual/Automatic	Manual/Automatic
Cost Model	**Pay-per-execution**	Pay 24/7	Pay 24/7
Best For	Event-driven, sporadic tasks	Complex, predictable apps	Monolithic apps

When to Choose Serverless?

- Variable/unpredictable traffic.
- Event-processing tasks.
- Low operational overhead.

When to Avoid?

- Long-running, constant processes.
- Strict low-latency needs (cold starts).
- Need for OS/runtime control.

Questions?